Build multiplataforma — Web, Android (APK) y Escritorio

La aplicación está construida con **React + Vite**, por lo que **ya es multiplataforma**: funciona en cualquier navegador (Windows, macOS, Linux, iOS y Android) y como **PWA** instalable. Además, con **Capacitor** se empaqueta como app **Android (APK)** nativa, y opcionalmente como app de **escritorio** con Electron.

Todo desde el mismo código fuente.

---

1. Web / PWA (sin pasos adicionales)

```
npm install
npm run build      # genera dist/ (incluye manifest.json y service worker)
npm run preview    # sirve el build localmente
```

El contenido de `dist/` se publica en cualquier hosting estático. La PWA permite "Instalar app" desde el navegador en móvil y escritorio.

> Recuerda crear `.env.local` con `VITE_BASE44_APP_ID` y `VITE_BASE44_APP_BASE_URL` (ver `README.md`) para que la app se conecte al backend.

---

2. Android (APK)

Opción A — Compilación automática en GitHub Actions (recomendada)

Ya hay un workflow listo: `.github/workflows/build-apk.yml`.

- En GitHub, ve a **Settings** → **Secrets and variables** → **Actions** y crea:
- `VITE_BASE44_APP_ID`
- `VITE_BASE44_APP_BASE_URL`
- Ve a la pestaña **Actions** → **Build Android APK** → **Run workflow** (o publica una etiqueta `vX.Y.Z`).
- Al terminar, descarga el **artefacto** `gestion-pro-total-debug-apk` (`app-debug.apk`) desde la página de la ejecución.

> El APK generado es de tipo **debug** (instalable activando "orígenes desconocidos"). Para distribución en Google Play se necesita un APK/AAB

> **firmado**: ver sección 2.C.

Opción B — Compilación local

Requisitos: **Node 20+**, **JDK 21**, **Android Studio** (o Android SDK + `ANDROID_HOME` configurado).

```
npm install
npm run build                 # 1) build web -> dist/
npm run cap:add:android       # 2) genera el proyecto nativo android/ (una sola vez)
npm run cap:sync              # 3) copia el build web al proyecto nativo
npm run cap:open:android      # 4) abre Android Studio para compilar/ejecutar

# …o construir el APK por línea de comandos:
npm run android:build         # genera android/app/build/outputs/apk/debug/app-debug.apk
```

Opción C — APK/AAB firmado para Google Play (automatizado)

Ya hay un workflow dedicado: **`.github/workflows/build-playstore-aab.yml`**.

A diferencia de `build-apk.yml` (que genera un APK *debug* para descarga directa), este compila un **AAB firmado con la clave de release real** y también un APK firmado para probarlo antes de subirlo — Play Store rechaza cualquier build firmado en modo *debug*.

- Genera una clave de firma (una sola vez, guárdala para siempre):

```bash
keytool -genkeypair -v -keystore release.keystore -alias gestionprototal \
-keyalg RSA -keysize 2048 -validity 10000
```

- Crea estos *secrets* en **Settings → Secrets and variables → Actions**:
- `ANDROID_KEYSTORE_BASE64` — el `.keystore` codificado en base64 (`base64 -w0 release.keystore`)
- `ANDROID_KEYSTORE_PASSWORD` — contraseña del keystore (y de la clave, es la misma)
- `ANDROID_KEY_ALIAS` — alias usado al generar la clave
- Ve a **Actions → Build Android AAB (Google Play) → Run workflow**.
- Descarga el artefacto `gestion-pro-total-playstore` (contiene `app-release.aab` y `app-release.apk`) y sube el `.aab` a Play Console.

El workflow toma automáticamente `versionCode` / `versionName` de la versión en `package.json` , así que cada subida a Play sube de versión sola.

> ⚠️ Guarda el `.keystore` y sus contraseñas fuera del repositorio (gestor de contraseñas, disco cifrado). Si se pierde, no se podrán publicar más actualizaciones sobre la misma ficha de Play Store.

---

3. iOS (opcional)

Requiere **macOS** con Xcode.

```
npm install @capacitor/ios
npm run build
npx cap add ios
npx cap sync ios
npx cap open ios     # compilar/firmar desde Xcode
```

---

4. Escritorio (Windows / macOS / Linux) con Electron

```
# Añadir Electron al proyecto (una vez)
npm install -D electron electron-builder

# Apuntar Electron a dist/ y empaquetar con electron-builder.
# Ver la guía oficial: https://www.electron.build
```

Como alternativa más integrada con Capacitor existe la comunidad `@capacitor-community/electron`.

---

5. Notas importantes

- **Variables de entorno**: el APK/escritorio incluye el build de `dist`, por lo que los valores `VITE_BASE44_*` quedan "horneados" en tiempo de compilación. Asegúrate de construir con los valores correctos del entorno de producción.
- **Iconos y splash**: para personalizarlos usa `@capacitor/assets` (`npx capacitor-assets generate`).
- **Permisos Android**: se editan en `android/app/src/main/AndroidManifest.xml`. Si necesitas persistir personalizaciones del proyecto nativo, deja de ignorar la carpeta `android/` en `.gitignore` y commitéala.
- **Por qué el APK no se genera en este repositorio directamente**: el entorno de desarrollo en la nube no tiene acceso de red a los servidores del SDK de Android

(Google), por eso la compilación del APK se delega a GitHub Actions, que sí dispone de SDK y red.